

Contents

1	¿Qué es HTTP(S)?	2
2	Estructura de un mensaje HTTP	2
3	Métodos	2

1 ¿Qué es HTTP(S)?

HTTP (Hypertext Transfer Protocol) es el protocolo encargado de la transferencia de información a través de archivos (XML, HTML, etc.). HTTP define la sintaxis y la semántica que utilizan los elementos de software de la arquitectura web (clientes, servidores, proxies) para comunicarse.

Es un protocolo orientado a transacciones y sigue el esquema petición-respuesta entre un cliente y un servidor. El **cliente** realiza una petición enviando un mensaje, con cierto formato al servidor. El **servidor** le envía un mensaje de respuesta.

2 Estructura de un mensaje HTTP

Los mensajes HTTP son en texto plano, lo que lo hace más legible y fácil de depurar. Los mensajes tienen la siguiente estructura:

- Línea inicial (termina con retorno de carro y un salto de línea) con:
 - * Para las **peticiones**: la acción requerida por el servidor (método de petición) seguido de la URL del recurso y la versión de HTTP que soporta el cliente.
 - * Para **respuestas**: la versión del HTTP usado seguido del código de respuesta (que indica qué ha pasado con la petición seguido de la URL del recurso) y de la frase asociada a dicho retorno.
- Las **cabeceras** del mensaje que terminan con una línea en blanco. Son metadatos. Estas cabeceras le dan gran flexibilidad al protocolo.
- Cuerpo del mensaje (*Opcional*). Su presencia depende de la línea anterior del mensaje y del tipo de recurso al que hace referencia la URL. Típicamente tiene los datos que se intercambian cliente y servidor. Por ejemplo, para una petición podría contener ciertos datos que se quieren enviar al servidor para que los procese. Para una respuesta podría incluir los datos que el cliente ha solicitado.

3 Métodos

HTTP define una **serie predefinida de métodos** de petición para indicar la acción que se desea realizar sobre el **recurso** identificado. Que este recurso sea preexistente (datos ya almacenados) o se genere de forma dinámica depende de la implementación del servidor. A menudo, el recurso corresponde a un archivo o a la salida de un ejecutable que reside en el servidor.

Un método puede ser catalogado como **seguro, idempotente o cacheable**:

- Método Seguro: Si una petición con ese método no tiene un efecto intencional en el servidor (**solo lectura**).
- Método idempotente: Si múltiples peticiones con ese método tienen el mismo efecto que una sola petición. Los métodos seguros son trivialmente idempotentes pues no deberían representar actualizaciones para el servidor.
- Método Cacheable: Si se pueden almacenar sus resultados para su reutilización.

Método	Petición con Carga Util	Respuesta con Carga Util	Safe	Idempotente	Cacheable
GET					
HEAD					
POST					
PUT					
DELETE					
CONNECT					
OPTIONS					
TRACE					
PATCH					